ASSIGNMENT-11.3

2303A51546

Batch-10

TASK-1:

PROMPT:

Generate a simple Python program for a Contact Manager using arrays (lists) that can add, search, and delete contacts with name and phone number.

CODE:

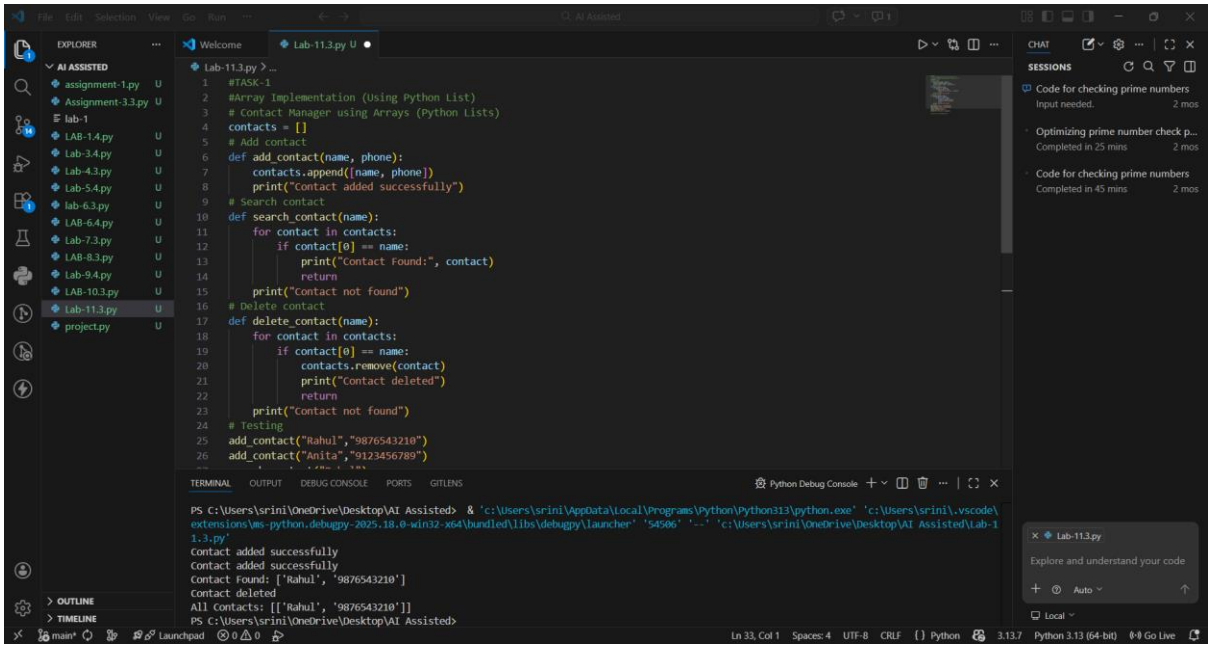
Array Implementation (Using Python List)

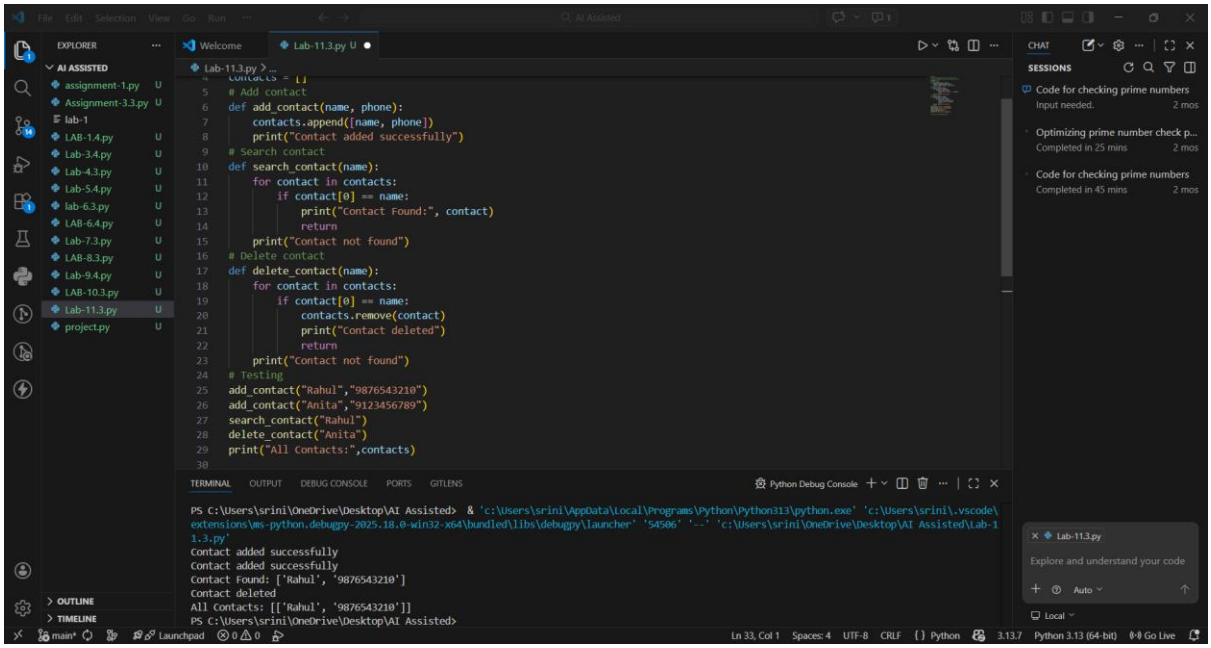
Contact Manager using Arrays (Python Lists)

```
contacts = []

# Add contact

def add_contact(name, phone):

    contacts.append([name, phone])

    print("Contact added successfully")

# Search contact

def search_contact(name):

    for contact in contacts:

        if contact[0] == name:

            print("Contact Found:", contact)

            return

    print("Contact not found")

# Delete contact

def delete_contact(name):

    for contact in contacts:
```

```
        if contact[0] == name:
            contacts.remove(contact)
            print("Contact deleted")
            return
    print("Contact not found")
# Testing
add_contact("Rahul","9876543210")
add_contact("Anita","9123456789")
search_contact("Rahul")
delete_contact("Anita")
print("All Contacts:",contacts)
```

OUTPUT:





2. Linked List Implementation

PROMPT:

Generate a Python program to implement a Contact Manager using a Linked List with add, search, and delete operations.

CODE:

 # Linked List Node

class Node:

  def __init__(self,name,phone):

    self.name = name

    self.phone = phone

    self.next = None

class ContactManager:

  def __init__(self):

    self.head = None

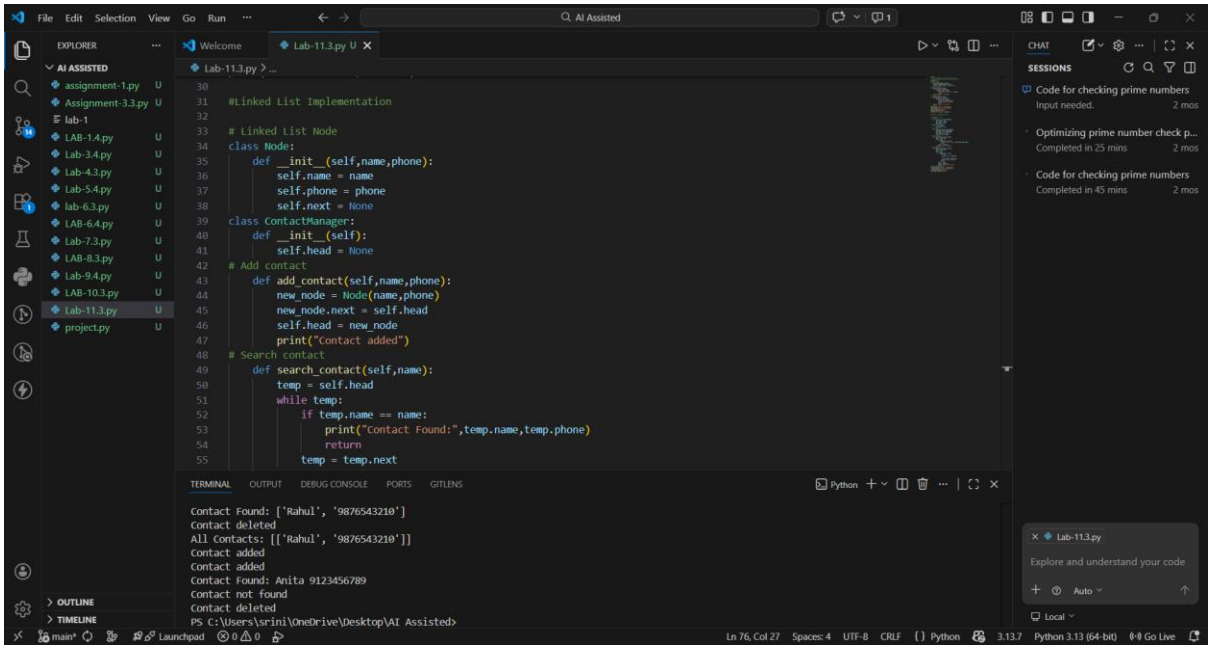
Add contact

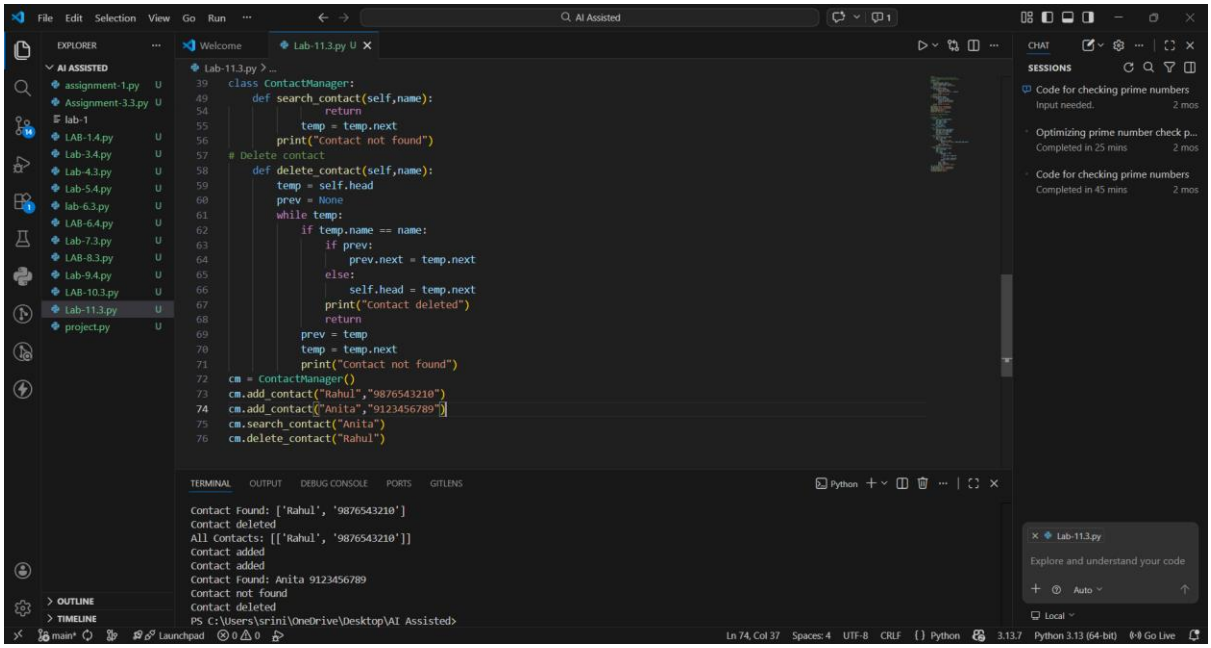
  def add_contact(self,name,phone):

```
        new_node = Node(name,phone)
        new_node.next = self.head
        self.head = new_node
        print("Contact added")
# Search contact
    def search_contact(self,name):
        temp = self.head
        while temp:
            if temp.name == name:
                print("Contact Found:",temp.name,temp.phone)
                return
            temp = temp.next
        print("Contact not found")
# Delete contact
    def delete_contact(self,name):
        temp = self.head
        prev = None
        while temp:
            if temp.name == name:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                print("Contact deleted")
                return
```

```
        prev = temp
        temp = temp.next
        print("Contact not found")
cm = ContactManager()
cm.add_contact("Rahul","9876543210")
cm.add_contact("Anita","9123456789")
cm.search_contact("Anita")
cm.delete_contact("Rahul")
```

OUTPUT:





Analysis:

Arrays store data in continuous memory locations.

Deleting elements takes more time because other elements must shift.

Linked lists store elements using nodes and pointers.

Linked lists allow easier insertion and deletion.

TASK-2:

PROMPT:

Create a Python program implementing Queue and Priority Queue for managing library book requests where faculty have higher priority than students.

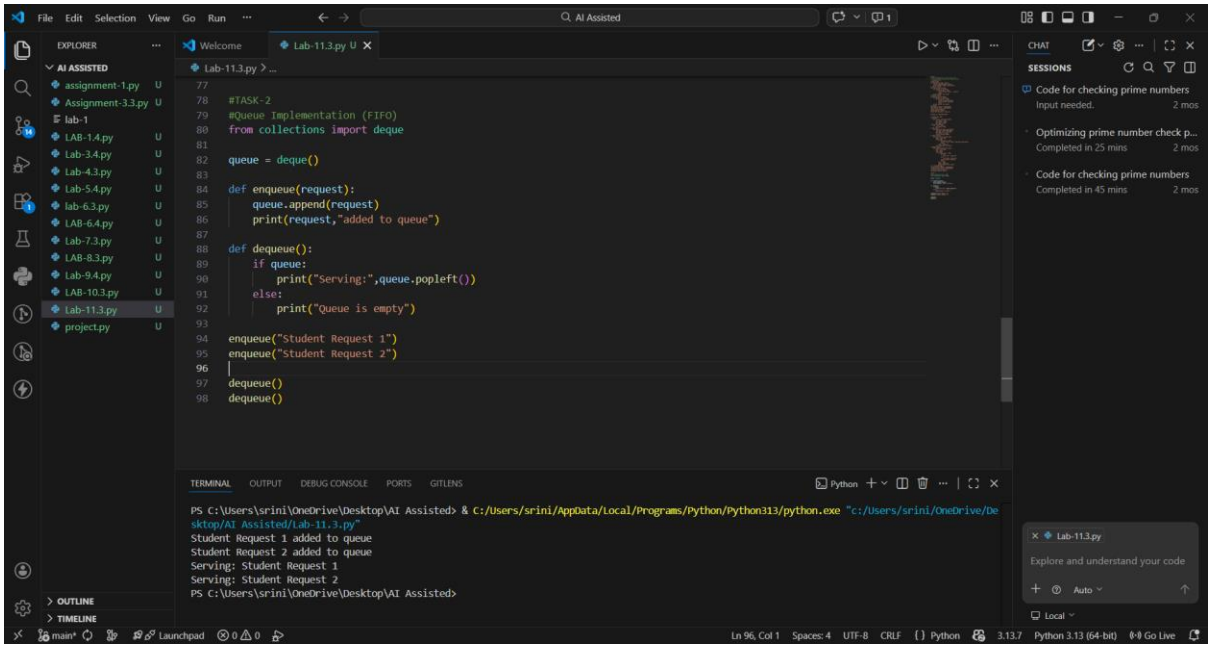
CODE:

#Queue Implementation (FIFO)

```
from collections import deque


queue = deque()


def enqueue(request):
```

```
    queue.append(request)
    print(request,"added to queue")

def dequeue():
    if queue:
        print("Serving:",queue.popleft())
    else:
        print("Queue is empty")

enqueue("Student Request 1")
enqueue("Student Request 2")

dequeue()
dequeue()
```

OUTPUT:

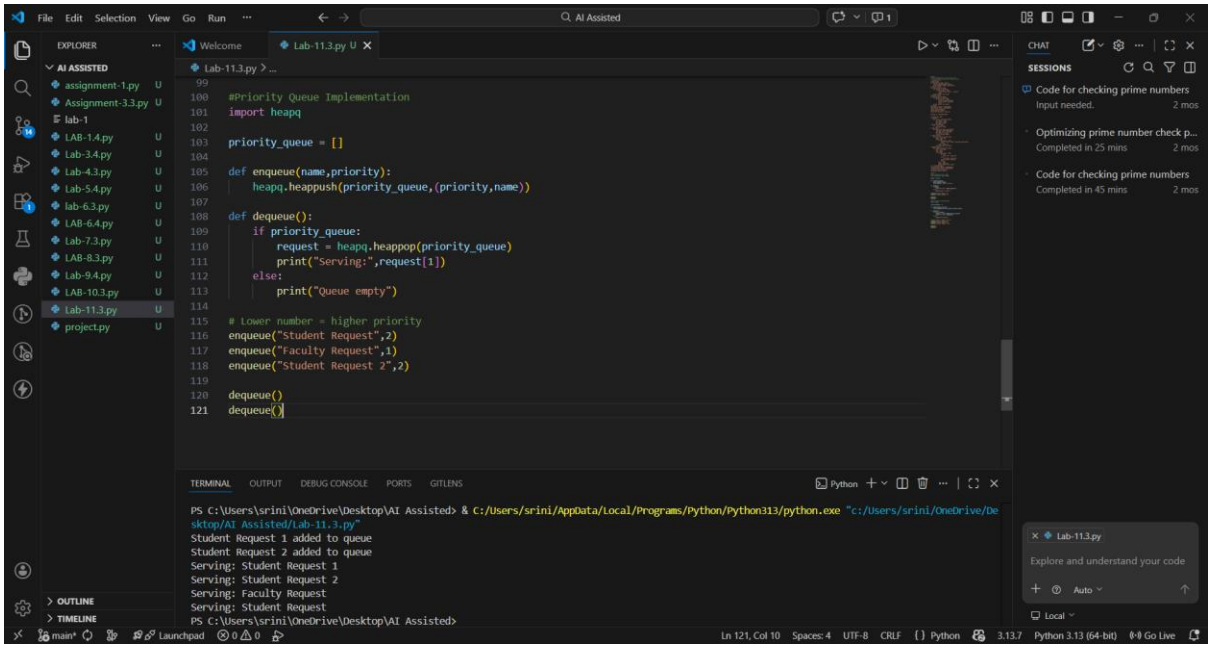


2. Priority Queue Implementation

CODE:

```
import heapq

priority_queue = []

def enqueue(name,priority):
    heapq.heappush(priority_queue,(priority,name))

def dequeue():
    if priority_queue:
        request = heapq.heappop(priority_queue)
        print("Serving:",request[1])
    else:
        print("Queue empty")

# Lower number = higher priority
enqueue("Student Request",2)
enqueue("Faculty Request",1)
enqueue("Student Request 2",2)

dequeue()
dequeue()
```

OUTPUT:



Analysis:

Queue follows FIFO (First In First Out).

Priority Queue processes requests based on priority level.

Faculty requests get higher priority than student requests.

TASK-3:

PROMPT:

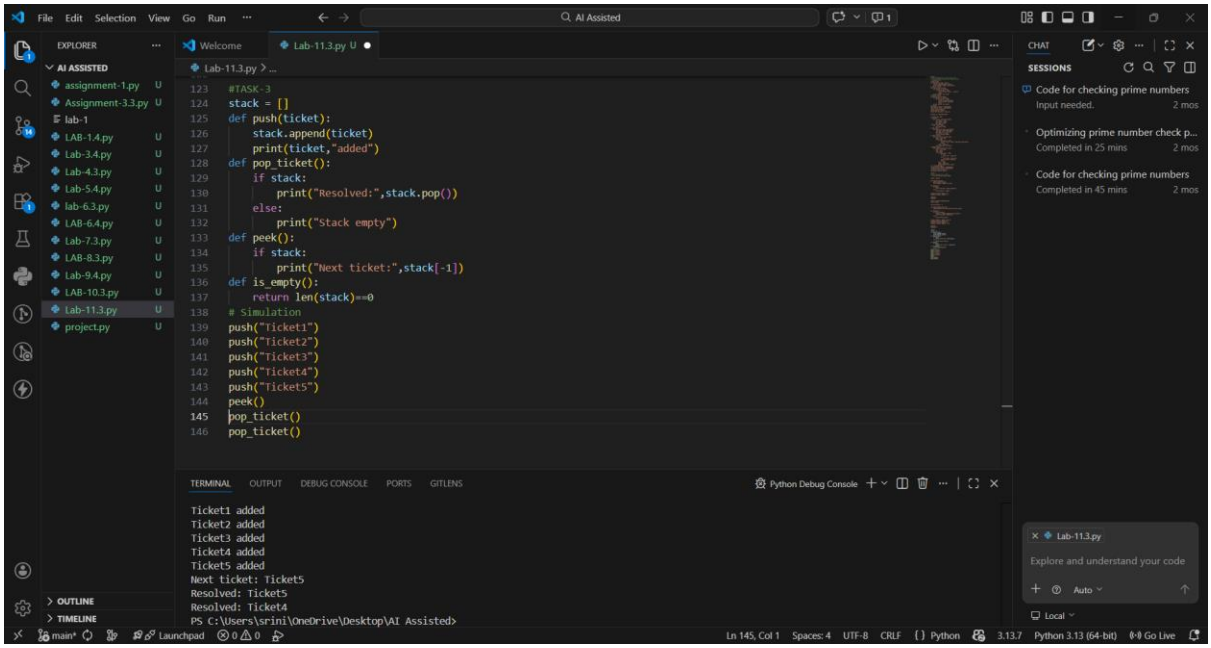
Generate a Python stack program for managing IT help desk tickets with push, pop, peek, is Empty operations.

CODE:

```
stack = []


def push(ticket):
    stack.append(ticket)
    print(ticket,"added")


def pop_ticket():
```

```
    if stack:
        print("Resolved:",stack.pop())
    else:
        print("Stack empty")

def peek():
    if stack:
        print("Next ticket:",stack[-1])

def is_empty():
    return len(stack)==0

# Simulation
push("Ticket1")
push("Ticket2")
push("Ticket3")
push("Ticket4")
push("Ticket5")

peek()

pop_ticket()
pop_ticket()
```

OUTPUT:



Analysis:

Stack follows LIFO (Last In First Out).
The most recently added ticket is resolved first.

TASK-4:

PROMPT:

Generate a Python hash table implementation with insert, search, delete and collision handling using chaining.

CODE:

```
class HashTable:


  def __init__(self,size):

    self.size = size

    self.table = [[] for i in range(size)]


  def hash_function(self,key):

    return key % self.size
```

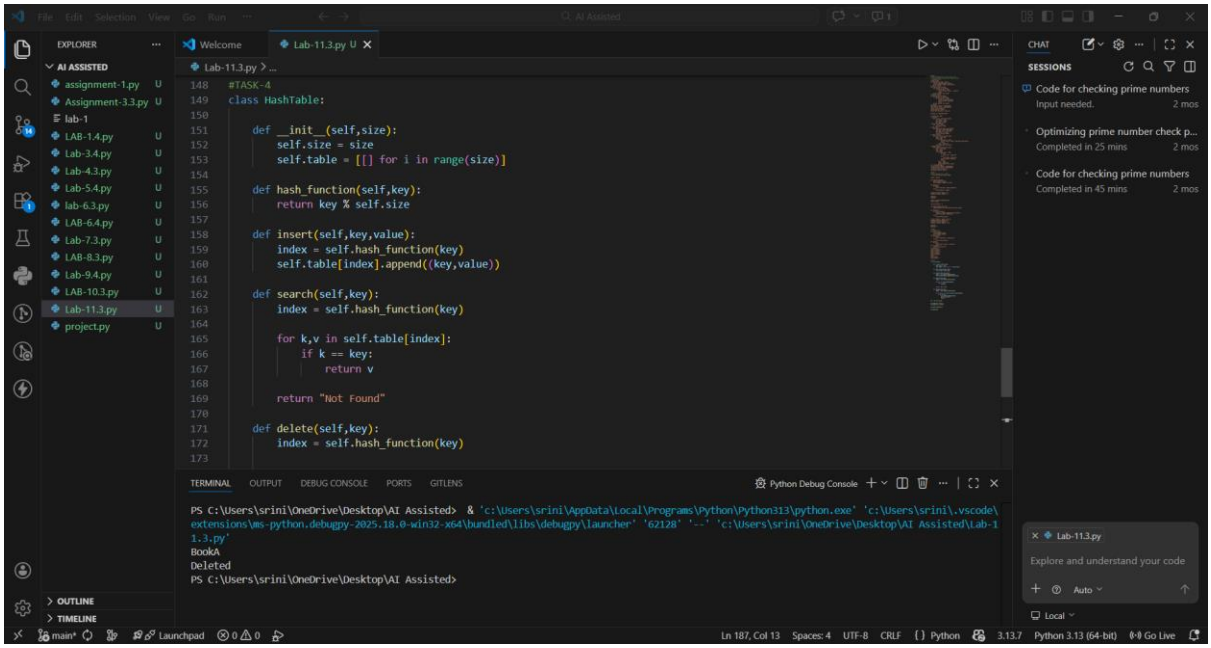
```
    def insert(self,key,value):
        index = self.hash_function(key)
        self.table[index].append((key,value))

    def search(self,key):
        index = self.hash_function(key)

        for k,v in self.table[index]:
            if k == key:
                return v

        return "Not Found"

    def delete(self,key):
        index = self.hash_function(key)

        for i,(k,v) in enumerate(self.table[index]):
            if k == key:
                self.table[index].pop(i)
                print("Deleted")
                return

ht = HashTable(10)

ht.insert(1,"BookA")
```

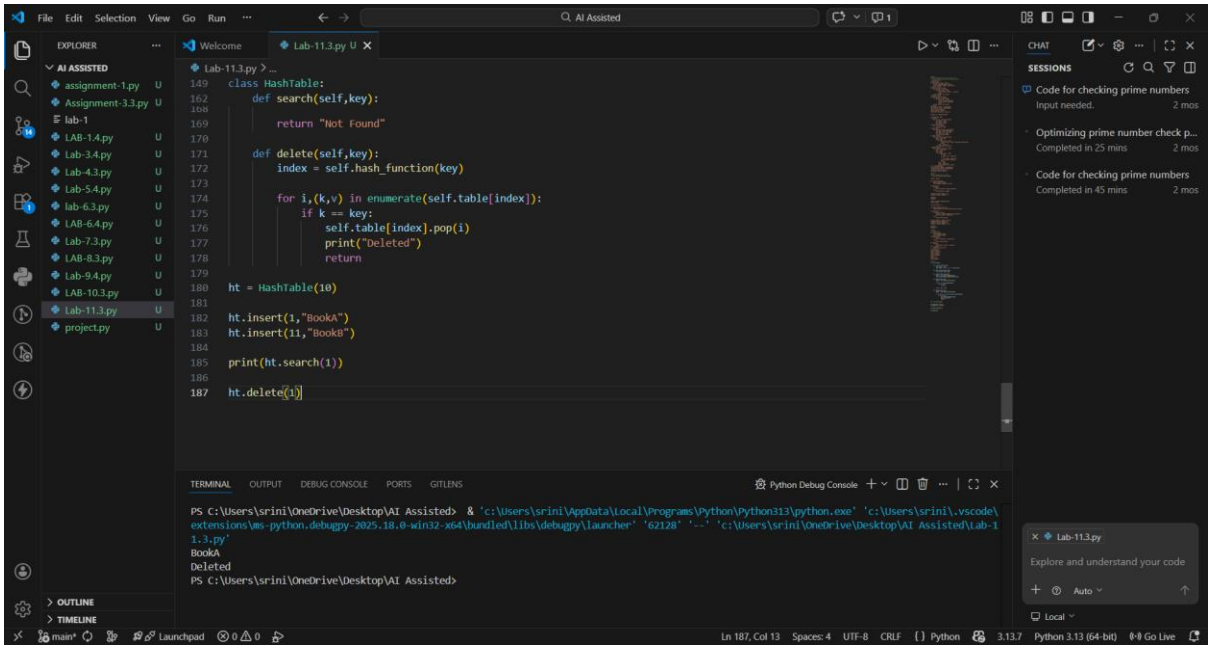
ht.insert(11,"BookB")

print(ht.search(1))

ht.delete(1)

OUTPUT:





Analysis:

Hash tables store data using key-value pairs.

The hash function determines the index.

When two keys map to the same index, chaining is used.

TASK-5:

PROMPT:

Generate a simple Python program using queue to manage cafeteria orders.

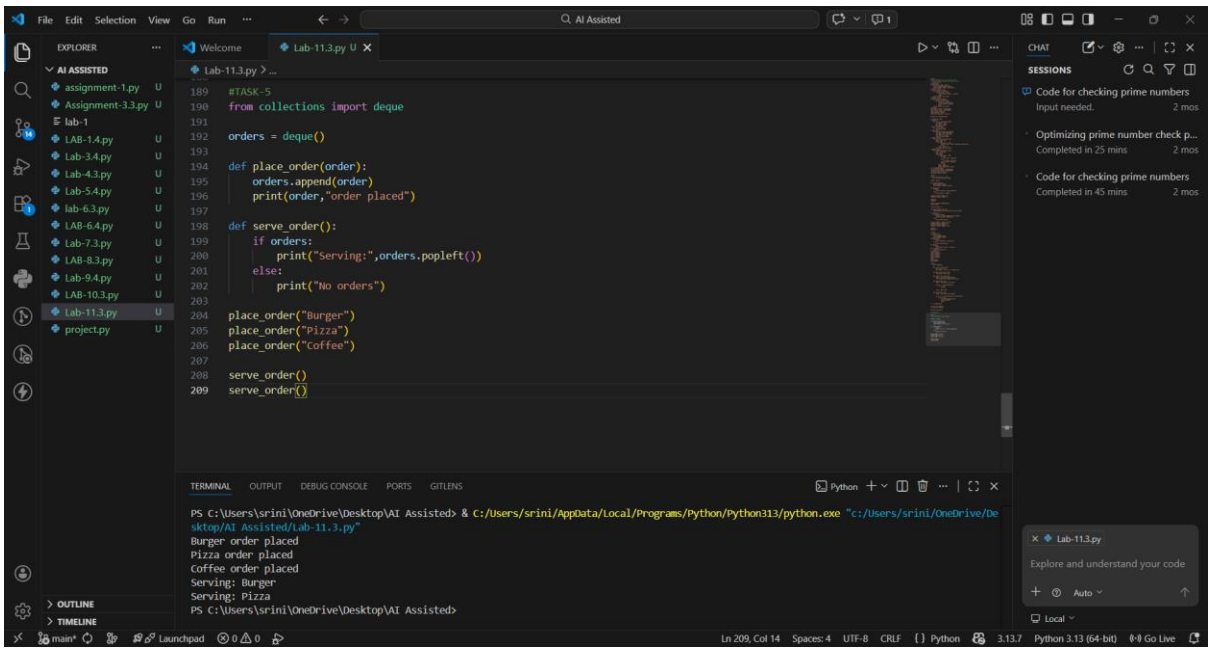
CODE:

```
from collections import deque


orders = deque()


def place_order(order):
    orders.append(order)
    print(order,"order placed")


def serve_order():
    if orders:
        print("Serving:",orders.popleft())
    else:
        print("No orders")


place_order("Burger")
place_order("Pizza")
place_order("Coffee")
```

serve_order()

serve_order()

OUTPUT:



Analysis:

Different data structures are used depending on the system requirements.

Hash tables provide fast searching.

Linked lists allow dynamic data management.

Queues help maintain order in requests and services.

Arrays are useful when the data size is fixed.

Using appropriate data structures improves the speed, organization, and efficiency of the system.